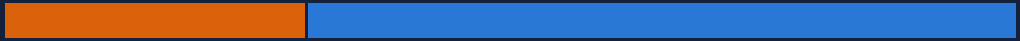
RÉPLICA DE UN PAPER · EURO-PAR 2024

A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting

Cómo leer un puntero compartido sin que otro hilo destruya el objeto a mitad de camino, y qué pasó al repetir los experimentos



peso · 19 bits

puntero · 45 bits

David Tamo Turpo · Jheremy Manuel Alvarez Astete · Diego Quispe Candelario

Paper de I. G. de Wolff, D. Anderson, G. K. Keller y A. Seletskiy

APERTURA

El recorrido: 15 partes y una pausa

Parte	Min	Expone
1 · Apertura	5	David
2 · El problema	10	David
3 · Vocabulario	8	David
4 · Los rivales	14	Jheremy
5 · La solución del paper	12	David
6 · Cómo se hizo	9	Jheremy
7 · Simular 4 Xeon	7	Jheremy
Pausa	10	—

Parte	Min	Expone
8 · Rendimiento	16	Jheremy
9 · Latencia	11	David
10 · Memoria y distancia	10	Diego
11 · Corrección	6	David
12 · Sin Docker	12	Diego
13 · Veredicto	9	Jheremy
14 · Defensa	7	Diego
15 · Recursos y cierre	5	Diego

Total: unas 2 h 30 min, más preguntas. Reparto sugerido a partir de RESPONSABILIDADES.md; ajústenlo a quien exponga cada parte.

APERTURA

Cada uno puso a prueba una afirmación del paper

David Tamo Turpo

COMPRUEBA

«Es correcto y wait-free» y «tiene la menor latencia de lectura»

EXPONE

Problema, vocabulario, solución del paper, latencia y corrección

Jheremy Manuel Alvarez Astete

COMPRUEBA

«Es el más rápido de los que no gastan memoria extra»

EXPONE

Rivales, método, simulación, rendimiento y veredicto

Diego Quispe Candelario

COMPRUEBA

«Sin sobrecoste de memoria» y la comparación entre versiones

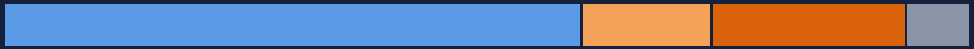
EXPONE

Memoria y distancia, sin Docker, defensa y recursos

Fuente: RESPONSABILIDADES.md (tabla del reparto) · repositorio davichiacustico-ux/replica-wait-free-refcount

EL RESULTADO, EN 20 SEGUNDOS

De las 15 afirmaciones del paper, la idea central sale bien parada



9 se reproducen

2 en parte

3 no se reproducen

1 no se confirma

Sale bien: es correcto, cumple su cota wait-free, no acumula basura y su lectura es la más rápida.

No sale: que gane a Folly, que gane con un millón de pilas, y la escala de 4 sockets, que este PC no tiene.

APERTURA

Siete tandas de medición, ninguna fallida

14 158

mediciones en total, con 0 fallidas

Tanda	Algoritmos	Mediciones
Con Docker, programas oficiales	Los 5 del paper	3 720
Sin Docker, programas oficiales	Los mismos 5	4 660
Sin Docker, bench.cpp del equipo	Autores, Libstdc++, Williams, Mutex	2 976
Sin Docker, bench.cpp alineado	Los mismos 4	2 160
Controles	Según el caso	546
Tráfico real de RAM	Los 5 y el techo de la máquina	70
Detectores de errores	Los 5 del paper y Mutex	26

Fuente: EXPLICACION.md, Parte 0 · CSV en replica/resultados/david-ultra9-285/

PARTE 2 · UNOS 10 MINUTOS

El problema

Leer un puntero compartido y sumar 1 a su contador son dos pasos. En medio, otro hilo puede destruir el objeto.

PARTE 2 · EL PROBLEMA

Cuatro ideas antes de empezar

Memoria

Cada dato vive en una dirección, como cada libro en un estante numerado.

Puntero

Un número que dice dónde está algo: una ficha que pone «estante 42».

Hilo

Una tarea que corre en un núcleo. Varios hilos son varias personas trabajando a la vez sin verse.

Use-after-free

Liberar demasiado pronto: alguien va al estante y encuentra otra cosa, o nada.

Fuente: EXPLICACION.md, Parte 1.1 · El paper cita el use-after-free como causa habitual de agujeros de seguridad (Chen et al., RAID 2023)

PARTE 2 · EL PROBLEMA

El conteo de referencias: una libreta pegada a cada libro

- **+1** Quien empieza a usar el libro suma 1 en su libreta.
- **−1** Quien termina resta 1.
- **0** Quien la deja en 0 era el último: tira el libro a la trituradora.

EN LOS LENGUAJES

`C++ · std::shared_ptr`

`Rust · Arc`

Con un solo hilo funciona perfecto. El lío empieza cuando hay varios.

Fuente: EXPLICACION.md, Parte 1.2

El tablón: un puntero que varios hilos leen y cambian a la vez

TABLÓN DEL VESTÍBULO

`estante 42`



LIBRO 42

`libreta = 1`

`load`

Leer el tablón y quedarte con una referencia al libro.

`store`

Cambiar el tablón para que señale otro libro.

Fuente: EXPLICACION.md, Parte 1.3 · En el paper: atomic shared pointer

La carrera, contada como historia

1. **Marta** lee el tablón: «estante 42». Va hacia allí.
2. Antes de que llegue, **Beto** cambia el tablón a «estante 77».
3. Beto resta 1 en la libreta del 42. Queda en 0: era el último.
4. Beto tira el libro 42 a la trituradora.
5. Marta llega y suma 1 en la libreta… de un libro que ya no existe.

SE LLAMA

Carrera read-reclaim

Una lectura compite con una liberación, y la lectura llega tarde.

Fuente: EXPLICACION.md, Parte 1.4 · Marta es un hilo lector (azul) y Beto, un hilo escritor (naranja)

La misma carrera, instante por instante

Instante	Marta (lee)	Beto (escribe)	Tablón	Libreta del 42
0			42	1
1	lee «42»		42	1
2		pone «77»	77	1
3		resta 1	77	0
4		tritura el libro	77	destruida
5	suma 1: use-after-free		77	destruida

El peligro está entre los instantes 1 y 5: Marta ya sabe dónde está el libro, pero todavía no lo ha reservado.

Fuente: EXPLICACION.md, Parte 1.5

En código, el agujero son dos líneas separadas

```
load:
p = tablon
  // ← aquí puede pasar todo el store
p.contador += 1
devolver p
```

```
store(nuevo):
viejo = intercambiar(tablon, nuevo)
viejo.contador -= 1
si viejo.contador == 0: liberar(viejo)
```

¿Por qué no hacer las dos cosas a la vez?

Porque el tablón y la libreta están en **dos sitios distintos** de la memoria.

Un procesador actual solo hace una operación indivisible sobre **un sitio** cada vez: es como tocar una sola cosa con la mano en cada instante.

Fuente: EXPLICACION.md, Partes 1.6 y 1.7 · pseudocódigo, no es el código real

Tres familias de solución

1

Poner un candado

Mientras alguien lee, nadie cambia nada. Simple, pero se forman colas.

Mutex · Libstdc++

2

Retrasar la trituradora

No se destruye nada hasta estar seguro de que nadie va de camino. Rápido, pero la basura se acumula un rato.

Anderson

3

Pagar por adelantado

Juntar puntero y cuenta en un solo sitio, para que leer y reservar sean el mismo gesto.

Williams · Folly · Autores

La aportación del paper: una versión de la familia 3 que además garantiza que **nadie se queda atascado**.

Fuente: EXPLICACION.md, Parte 1.7

PARTE 3 · UNOS 8 MINUTOS

Vocabulario

Cuatro ideas que explican casi todas las gráficas: atómicas, garantías de progreso, caché y cómo leer los números.

Atómica: ocurre entera o no ocurre

Operación	Qué hace	Analogía
fetch_add / fetch_sub	Suma o resta y devuelve el valor anterior	Arrancar un papelito del taco de turnos
exchange	Pone un valor nuevo y devuelve el viejo	Cambiar el cartel y quedarte con el antiguo
CAS	«Si vale X, ponlo a Y; si no, dime cuánto vale»	Cambio el cartel solo si sigue como lo leí
DWCAS	Un CAS sobre 16 bytes en vez de 8	Lo mismo sobre un cartel doble; más lento

`fetch_add` nunca falla

Pides tu turno y lo tienes, aunque haya mil personas.

`CAS` puede fallar

Si alguien cambió el valor justo antes, reintentas. Con mucha gente, pierdes una y otra vez.

Fuente: EXPLICACION.md, Parte 2.1 · CAS = compare-and-swap; DWCAS = double-width CAS (instrucción cmpxchg16b)

Tres garantías de progreso, de menos a más

Bloqueante

Una sola llave. Si quien la tiene se duerme, **todos** esperan.

«Depende de los demás»

Lock-free

Puerta giratoria a empujones: siempre pasa alguien, pero **tú** podrías perder siempre.

«Depende de la suerte»

Wait-free

Cada uno tiene su torniquete: pase lo que pase, terminas en N pasos.

«No depende de nadie»

Wait-free no significa más rápido. Significa más predecible.

Fuente: EXPLICACION.md, Parte 2.2 · Herlihy, «Wait-free synchronization», ACM TOPLAS, 1991

Compartir es caro: el cuaderno que viaja de mano en mano

Cada núcleo tiene una **caché**: memoria pequeña y rapidísima. Los datos viajan en **líneas de 64 bytes**.

Para escribir en una línea, un núcleo debe tenerla **en exclusiva**: hay que quitársela al que la tenía.

Si todos escriben en la misma variable, añadir hilos no acelera nada. Eso es la **contención**.

LA ANALOGÍA

Veinte personas deben escribir en **un único cuaderno**, que no se puede fotocopiar.

Escribir dura un instante. Lo que consume el tiempo es pasarse el cuaderno.

Fuente: EXPLICACION.md, Parte 2.3

Dos trampas más: la distancia y la compartición falsa

La distancia

Pasar el cuaderno al compañero de mesa es rápido; mandarlo a otro edificio, lento.

Con varios **sockets** (varios chips en una placa), mandar una línea a otro chip es varias veces más lento. El paper usa **4**.

Compartición falsa

Dos datos **distintos** que caen en la misma línea se estorban, aunque nadie los comparta.

Es escribir en renglones distintos de la misma hoja. En esta réplica costó hasta **3,65 veces** de rendimiento.

Fuente: EXPLICACION.md, Partes 2.3 y 2.4 · la compartición falsa se cuenta entera en la parte 12

Rendimiento y latencia no son lo mismo

Mop/s

Millones de operaciones por segundo: el **rendimiento**. Más es mejor.

ns · µs

Cuánto tarda una operación: la **latencia**. Menos es mejor. 1 µs = 1 000 ns.

p50

La mediana: la mitad fue más rápida. Es la operación **típica**.

p99

El 99 % fue más rápido: lo **malo** que puede ponerse. Mediana buena y p99 horrible es ir a tirones.

Fuente: EXPLICACION.md, Parte 2.4

PARTE 4 · UNOS 14 MINUTOS

Los rivales

De lo más simple a lo más sofisticado: la versión rota, el candado, Libstdc++, Williams, Folly y Anderson.

PARTE 4 · LOS RIVALES

La versión ingenua: la que está rota a propósito

Leer el puntero y, después, sumar 1 al contador.

Es exactamente la carrera de la parte 2. No es un rival del paper: es el problema. Está en `codigo/estres_asan.cpp` para demostrar que el fallo existe y que el detector lo ve.

LO QUE ENCONTRARON LOS DETECTORES

ASan (8, 12 y 16 hilos): double-free; la ejecución se para enseguida

TSan (8 y 16 hilos): carreras de datos y heap-use-after-free

Boceto didáctico: 227 031 errores

Fuente: EXPLICACION.md, Partes 3.0 y 6.5 · resultados/david-ultra9-285/correccion/

Mutex: una sala con una sola llave

El tablón está en una sala cerrada. Si la llave está ocupada, **te sientas a dormir** y te despiertan cuando quede libre.

`load`

1. Pedir la llave; si no está libre, el sistema te duerme
2. Leer el puntero
3. Sumar 1 al contador: estás solo
4. Devolver la llave

GARANTÍA

Bloqueante

MEMORIA EXTRA

Ninguna; libera al instante

A FAVOR

Muy simple y correcta; dormir no gasta CPU

EN CONTRA

Colas; dormir y despertar es lento, y su p99 es malísimo

Fuente: EXPLICACION.md, Parte 3.1 · solo en bench.cpp del equipo; no está en el paper

Libstdc++: un pestillo dentro del propio puntero

Los objetos están alineados, así que el último bit del puntero siempre vale 0. GCC lo usa de pestillo: **0 libre, 1 ocupado**. Quien no entra se queda de pie, dando vueltas.

`load`

1. Mirar el bit; si está a 1, esperar dando vueltas
2. Ponerlo a 1 con un CAS; si otro se adelantó, volver a 1
3. Leer el puntero y sumar 1 al contador
4. Restar 1 a la palabra para apagar el bit

GARANTÍA

Bloqueante

MEMORIA EXTRA

Ninguna; libera al instante

A FAVOR

Viene con el compilador; sin contención es muy ágil

EN CONTRA

Cada lectura escribe dos veces en la línea más disputada

Fuente: EXPLICACION.md, Parte 3.2 · std::atomic<std::shared_ptr> de GCC, C++20 (bits/shared_ptr_atomic.h)

Williams: apuntarse en el tablón antes de ir al libro

El tablón tiene dos columnas: la ficha del libro y una rayita por cada persona que **va de camino**. Contador partido: una parte junto al puntero y otra en la libreta. Son **16 bytes**, con DWCAS.

`load`

1. Bucle de DWCAS: hacer mi rayita
2. Sumar 1 en la libreta: la rayita me protege
3. Bucle de DWCAS: borrar mi rayita
4. Si el puntero cambió, avisar en la libreta

GARANTÍA

Lock-free

MEMORIA EXTRA

Ninguna; libera al instante

A FAVOR

Sin candados: nadie bloquea a nadie

EN CONTRA

Cada lectura toca el tablón dos veces, con DWCAS

Fuente: EXPLICACION.md, Parte 3.3 · Williams, «C++ Concurrency in Action», 2012 · split reference count

Folly: Williams en 8 bytes, con provisión de lectores

Los punteros reales usan 48 de sus 64 bits; en los 16 libres cabe el contador local. Cada libro trae anotada una **provisión de 8 192 usuarios**, así que casi nunca hay que ir a la libreta.

`load`

1. Bucle de CAS para sumar 1 al contador local
2. Si pasa de 4 096, trasladar un lote a la libreta
3. Si llegara a 8 192, esperar cediendo el turno

GARANTÍA

Lock-free; bloqueante con más de 8 192 hilos

MEMORIA EXTRA

Ninguna; libera al instante

A FAVOR

Una operación de 8 bytes por lectura

EN CONTRA

CAS en bucle: puede perder y reintentar

Fuente: EXPLICACION.md, Parte 3.4 · Facebook/Meta Folly, AtomicSharedPtr.h (takeOwnedBase)

PARTE 4 · LOS RIVALES · FAMILIA 2

Anderson: un cartel en tu mesa y una caja de pendientes

Antes de ir al libro, escribes «42» en tu cartelito: un **hazard pointer**. Quien libera mete la libreta en una caja y, cada **2 000** retiradas, revisa todas las mesas.

`load`

1. Leer el tablón y copiarlo en mi cartelito
2. Releer el tablón; si cambió, repetir
3. Sumar 1 si el contador no es 0; si lo es, repetir
4. Borrar mi cartelito

GARANTÍA

Lock-free

MEMORIA EXTRA

Sí: bloques de control pendientes

A FAVOR

Leer no escribe en el tablón

EN CONTRA

Memoria retenida y picos de latencia al vaciar la caja

Fuente: EXPLICACION.md, Parte 3.5 · Anderson, Blelloch y Wei, PLDI 2021 y 2022 · Michael, hazard pointers, IEEE TPDS 2004

Los seis, de un vistazo

Algoritmo	Atómicas por lectura	¿Reintenta?	Garantía	Liberación	¿Leer escribe?
Mutex	Candado + desbloqueo	Espera dormido	Bloqueante	Inmediata	Sí
Libstdc++	CAS + fetch_sub	Sí, dando vueltas	Bloqueante	Inmediata	Sí
Williams	2 bucles de DWCAS	Sí	Lock-free	Inmediata	Sí
Folly	1 bucle de CAS	Sí	Lock-free	Inmediata	Sí
Anderson	Ninguna en el tablón	Sí, pocas veces	Lock-free	Diferida	No
Autores	1 fetch_sub	No (salvo recarga, $\leq$ P)	Wait-free	Inmediata	Sí

En Autores, con una sola pila, hay **dos** líneas calientes: el tablón y la libreta. En Anderson, solo la libreta.

Fuente: EXPLICACION.md, Parte 3.7 · «atómicas por lectura» se refiere a la zona disputada, el tablón

PARTE 5 · UNOS 12 MINUTOS

La solución del paper

El tablón guarda un taco de vales prepagados. Arrancar un vale te da la dirección del libro en un solo gesto indivisible.

La historia de los vales

1

Antes de poner el libro

El bibliotecario anota en la libreta «hay 100 usuarios» y deja **100 vales** en un taco pegado al tablón.

2

Marta arranca un vale

El vale lleva impreso «estante 42». En un gesto lee la dirección y se lleva una reserva pagada. **Ni toca la libreta.**

3

Beto cambia el tablón

Se lleva el taco con los 97 que quedan y los resta. Quedan 3: los que la gente tiene en la mano.

Cada uno resta 1 al terminar. El último deja la libreta en 0 y tira el libro **en ese mismo instante**: sin caja de pendientes.

Fuente: EXPLICACION.md, Parte 3.6 · los 100 vales son un número de ejemplo

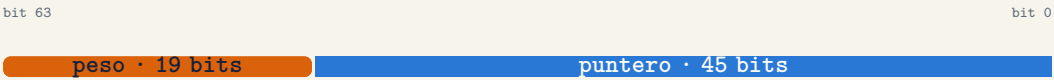
LA REGLA DE ORO

Libreta = vales en el taco + vales en manos de la gente

Mientras se cumpla, nunca llega a 0 algo que alguien pueda usar. En el paper, cada vale es un **peso** y el sistema, **conteo de referencias ponderado**.

PARTE 5 · LA SOLUCIÓN DEL PAPER

Todo cabe en una palabra de 64 bits



POR QUÉ 45

Los punteros reales usan 48 bits, y los 3 últimos siempre son 0 por alineación.

CUÁNTOS VALES

Quedan 19 bits: hasta **C = $2^{19} - 1$ = 524 287** vales en el taco.

POR QUÉ IMPORTA

Un solo fetch_sub sobre esta palabra lee la dirección y arranca el vale a la vez.

Fuente: EXPLICACION.md, Parte 3.6 · la barra está a escala: 19 de 64 bits en naranja

Un ejemplo con números pequeños: C = 10, P = 3

Paso	Quién	Acción	Tablón	Libreta de A	Libreta de B
0		Situación inicial	(A, 10)	10	
1	Marta	load: fetch_sub del peso	(A, 9)	10	
2	Luis	load: fetch_sub del peso	(A, 8)	10	
3	Beto	Prepara B: suma C a su libreta	(A, 8)	10	10
4	Beto	exchange: pone B, se lleva (A, 8)	(B, 10)	10	10
5	Beto	Resta a A los 8 vales sobrantes	(B, 10)	2	10
6	Marta	Termina: resta 1	(B, 10)	1	10
7	Luis	Termina: resta 1	(B, 10)	0: se destruye	10

Paso 2: 8 en el taco + 2 en mano = 10. Paso 5: 0 en el taco + 2 en mano = 2. La regla de oro se cumple siempre.

Fuente: EXPLICACION.md, Parte 3.6 · cumple la regla $C \geq 2P$ que se explica más adelante

load y store: tres pasos, sin bucles

load

1. fetch_sub: quitar 1 al peso. Devuelve puntero y peso. Nunca falla
2. Si el peso sigue $\geq$ P, fin. Es casi siempre
3. Si bajó de P, recargar con increase_weight

store

1. Sumar C $-$ 1 a la libreta del objeto nuevo
2. exchange: poner (nuevo, C) y quedarse con (viejo, peso)
3. Restar el peso a la libreta del viejo; si llega a 0, se destruye ya

Sin bucles: **wait-free y de tiempo constante**. El camino rápido de load es una sola instrucción atómica.

Fuente: EXPLICACION.md, Parte 3.6 · el binario oficial usa una versión espejo (tag, fetch_add) de la del repositorio (weight, fetch_sub)

La recarga reintenta como mucho P veces

increase_weight suma los vales a la libreta e intenta con un CAS subir el peso del tablón. Si el CAS falla, mira **por qué**:

Lo que ve al fallar	Qué significa	Qué hace
El puntero ya es otro	Beto cambió el tablón	Deshace y sale
El peso subió	Otro hilo ya recargó	Deshace y sale: lo nuevo
El peso bajó	Otro hilo arrancó un vale	Reintenta con el peso nuevo

Solo reintenta si el peso **bajó**. Empezó por debajo de P y nunca baja de 0: como mucho **P reintentos**, sin depender de la suerte.

Fuente: EXPLICACION.md, Parte 3.6 · demostración formal en el paper, secciones 4.1 y 4.2

PARTE 5 · LA SOLUCIÓN DEL PAPER

Dos reglas para que nunca se acaben los vales

R-1 · C ≥ 2P

El taco completo tiene al menos el doble de vales que hilos.

R-2

El peso solo cambia bajando de 1 en 1 o saltando a un valor mayor que P.

DICHO CON PERSONAS

Cada hilo solo puede arrancar **un vale de más** antes de ponerse a recargar. Con P hilos, el peso baja como mucho P por debajo del umbral P: **P − P = 0**. Nunca es negativo.

Fuente: EXPLICACION.md, Parte 3.6 · en la parte 11 se rompe R-1 a propósito: 42 348 errores

PARTE 6 · UNOS 9 MINUTOS

Cómo se hizo

Los programas oficiales de los autores, su receta exacta, otra máquina y cuatro adaptaciones declaradas.

El paper mide dos cosas, en dos máquinas distintas

FIGURA 5 · RENDIMIENTO

4 Xeon E7-8867: 72 núcleos, 144 hilos

n pilas. Cada operación elige una al azar: con probabilidad *u* saca un elemento y lo mete en otra; si no, lee la cima. 6 combinaciones de (*n*, *u*).

FIGURA 6 · LATENCIA

Xeon w7-3465X: 28 núcleos, 56 hilos

p hilos a la vez: el primero se mide y los demás hacen store sin parar. Se miden load, store y store que libera.

n pequeño = mucha gente en poco sitio

u grande = mucha escritura

Fuente: EXPLICACION.md, Parte 4.1 · los valores del paper se extrajeron de su gráfico vectorial con herramientas/extraer_fig5.py

El código de los autores, con su receta exacta

Artefacto oficial de Zenodo, **MD5 comprobado**, en la misma imagen base: Ubuntu 22.04 y g++-12.

No es una reimplementación: es su código, el de Folly y el de Anderson, y su programa de pruebas.

10.5281/zenodo.11775182

Detalle	Valor
Orden	bench_stack -r 10 -s n -u u -i 7 –peek=true
Algoritmos	ivo, gnu, jss-free, folly, mine
Asignador	jemalloc con LD_PRELOAD
Memoria	numactl -i all
Hilos	1, 18, 36, 54, 72, 108, 144, 180, 216
Tamaño de pila	20 elementos + T/n
Resultado	Mediana de 7 ejecuciones de 10 s

Fuente: EXPLICACION.md, Parte 4.2 · ivo = Autores, gnu = Libstdc++, jss-free = Williams, mine = Anderson

La máquina del paper frente a la nuestra

	Paper, figura 5	Réplica
Procesador	4 × Intel Xeon E7-8867	1 × Intel Core Ultra 9 285
Núcleos	72	24: 8 P (rápidos) + 16 E (eficientes)
Hilos de hardware	144 (con SMT)	24 (sin SMT)
Sockets y NUMA	4 sockets, 4 nodos	1 socket, 1 nodo
RAM	Repartida entre 4 chips	Un módulo de 32 GB DDR5-5600
Caché L3	—	36 MB

Esta CPU es casi diez años más moderna. Por eso no se comparan Mop/s absolutos: se comparan **órdenes y proporciones**.

Fuente: EXPLICACION.md, Partes 5.2, 6 y 8 · DEFENSA.md, chuleta de cifras

Cuatro adaptaciones, todas declaradas

Paper	Aquí	Por qué
Hilos 1 a 216	1, 3, 6, 9, 12, 18, 24, 30, 36	La lista del paper entre 6: 24 es «toda la máquina», como 144; 36 es 1,5 veces, como 216
7 × 10 s, un proceso	5 × 2 s, un proceso por punto, en orden aleatorio	La figura 5 pasa de unas 5 h a poco más de 1 h; el calor o una interrupción afectan a todos por igual
p = 56	p = 1, 24 y 56	24 son todos los núcleos reales; 56 se mide, pero no es comparable
Williams fuera de la figura 6	Sí se midió	El paper lo omite «porque su latencia casi siempre era mayor»: se comprobó en vez de creerlo

Y como recortar puede sesgar, se repitieron puntos con la receta **exacta** para medir cuánto cambia (parte 12).

Fuente: EXPLICACION.md, Partes 4.3 y 6.7

¿Réplica o reproducción? La ACM distingue tres palabras

Palabra	Quién	Con qué	¿Es lo nuestro?
Repetición	El mismo equipo	Su propio montaje	No
Reproducción	Otro equipo	Los artefactos de los autores	Sí: programas oficiales, otra máquina
Réplica	Otro equipo	Artefactos propios	En parte: bench.cpp es nuestro

En el curso se llama «réplica», y está bien. Si el jurado dice «esto es una reproducción», **tiene razón**, y bench.cpp añade una segunda opinión con un programa distinto.

Fuente: DEFENSA.md, sección 2 · ACM, «Artifact Review and Badging», versión vigente

PARTE 7 · UNOS 7 MINUTOS

Simular 4 Xeon con una sola CPU

No con una máquina virtual, sino midiendo las distancias reales que ya existen dentro del chip.

Qué significa de verdad tener 4 sockets

1

Distancia entre núcleos

Pasar una línea de caché a otro chip es varias veces más lento. **Esto sí se puede imitar.**

2

Distancia a la memoria

NUMA: cada chip tiene su RAM y la del vecino está más lejos. **Esto no.**

DESCARTADO

Una máquina virtual

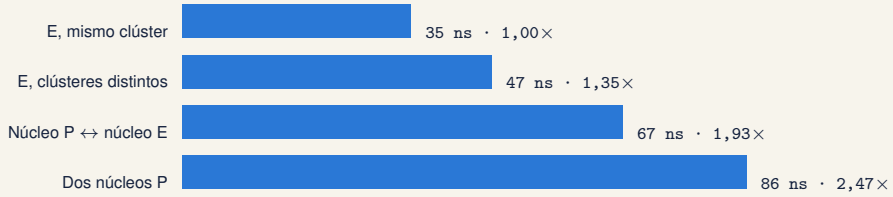
Le dice al sistema que hay 4 nodos, pero ningún viaje tarda más. Sería un decorado.

Fuente: EXPLICACION.md, Parte 5.1

PARTE 7 · SIMULAR 4 XEON

Medir antes de suponer: los 4 clústeres sí se comportan como sockets

Tiempo que tarda una línea de caché en viajar entre dos núcleos (mediana por clase)



Sorpresa: los núcleos P, los más rápidos calculando, son los más lentos pasándose datos.

Pero más suave: en Xeon de esa época, entre sockets ronda 3×; aquí, entre clústeres, 1,35×.

Fuente: tablas.md, «Latencia entre núcleos» · herramientas/latencia_nucleos.cpp · figura topologia.png · 3×: Intel Community y Chips and Cheese, dato indirecto

La batería de afinidad: fijar cada hilo a un núcleo concreto

Colocación	Núcleos	Qué imita
E-compacto	Llena un clúster y luego el siguiente	Todos en el mismo socket
E-disperso	Reparte entre los 4 clústeres	Hilos en los 4 sockets
P	Núcleos P	Máxima distancia, 86 ns
todos24	Los 24, fijados	Toda la máquina
sistema	Sin fijar	Lo que hizo el paper

DOSIS-RESPUESTA

Con 4 hilos, solo cambia la distancia:

$35 \rightarrow 47 \rightarrow 86$ ns

Subimos la dosis y miramos cuánto cae cada algoritmo.

El binario oficial no se toca: fijar_hilos.c se carga con LD_PRELOAD e intercepta la creación de hilos.

Fuente: EXPLICACION.md, Parte 5.4 · herramientas/fijar_hilos.c · listas de núcleos en baterias.py

PAUSA · 10 MINUTOS

Lo que ya sabemos

1. Leer y reservar son dos pasos, y en medio cabe una liberación.
2. El paper lee con un solo fetch_sub que nunca falla, y recarga con una cota de P intentos.
3. Medimos su código oficial, en otra máquina, y simulamos la distancia midiéndola.

PARTE 8 · UNOS 16 MINUTOS

Rendimiento

La figura 5: qué se repite, el misterio de Folly y por qué con un millón de pilas todos empatan.

Tres reglas antes de mirar ninguna gráfica

1. **Se comparan algoritmos, no máquinas.** Importan el orden y la proporción, no los Mop/s absolutos.

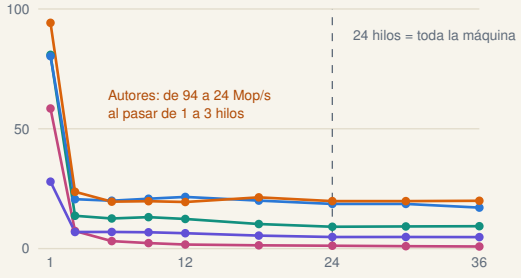
2. **«0,50×» significa «rinde la mitad que Autores»**, y «2×», el doble. Así se compara sin que importe la máquina.

3. **Una diferencia pequeña no es una victoria.** Si los rangos de las rondas se solapan, o la diferencia es de pocos puntos, es **empate**.

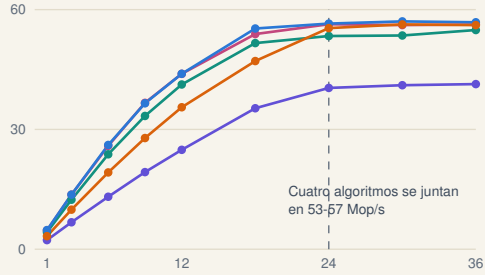
Fuente: EXPLICACION.md, Parte 6 · calidad con Docker: 2 700 ejecuciones de rendimiento, 0 fallidas, dispersión típica entre rondas del 1 % (8 % en los puntos más ruidosos)

Una pila: todo se desploma. Un millón: todo sube

n = 1, u = 1 % · Mop/s según los hilos



n = 10^6, u = 50 % · Mop/s según los hilos





Fuente: docker/rendimiento.csv, mediana de 5 rondas por punto · figura completa con los 6 escenarios: figuras/fig5_replica.png

El marcador con toda la máquina: coincide en 4 de 6

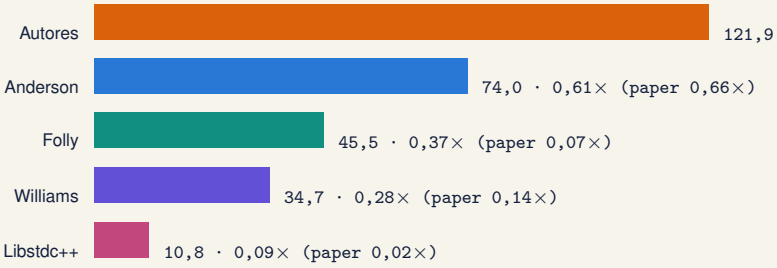
Escenario	En palabras	Paper (144 hilos)	Réplica (24 hilos)	¿Coincide?
n = 1, u = 1 %	Una pila, casi todo leer	Anderson 1,70×	Autores, por poco (Anderson 0,94×)	No
n = 1, u = 10 %	Una pila, algo de escribir	Anderson 1,66×	Anderson 1,99×	Sí
n = 1, u = 50 %	Una pila, mucho escribir	Anderson 1,82×	Anderson 1,87×	Sí
n = 10, u = 1 %	10 pilas, casi todo leer	Autores (Anderson 0,66×)	Autores (Anderson 0,61×)	Sí, casi exacto
n = 10, u = 10 %	10 pilas, algo de escribir	Anderson 1,54×	Anderson 1,13×	Sí, por menos
n = 10^6, u = 50 %	Un millón de pilas	Autores	Empate a cuatro, unos 55 Mop/s	No

τ de Kendall entre el orden del paper y el nuestro: `+0,60 +0,60 +0,40 +0,80 +0,80 +0,20`. Siempre positiva.

Fuente: EXPLICACION.md, Parte 6.1 · tablas.md, «Rendimiento: paper (144 hilos) frente a réplica (24 hilos)» · figura fig5_paper_frente_a_replica.png

El mejor caso de Autores se reproduce casi exacto

n = 10 pilas, u = 1 % de escrituras, 24 hilos · Mop/s, mediana de 5 rondas



Pocas escrituras y algo de reparto: el terreno donde un único fetch_sub que nunca falla brilla.

Fuente: tablas.md, «Rendimiento: paper (144 hilos) frente a réplica (24 hilos)» · EXPLICACION.md, Parte 6.1

Lo que se repite

Libstdc++ se hunde

Con n = 1 y u = 1 %, de 58,5 a 1,2 Mop/s: unas **49 veces** menos. En el paper, de 7 a 0,11: unas 64.

Es lo que hace un candado: formar cola.

Williams, siempre debajo

Por debajo de Autores en los **6 escenarios**, entre 0,24× y 0,73×.

Dos bucles de DWCAS por lectura se pagan siempre.

Anderson, con escrituras

Con una pila y u $\geq$ 10 %, rinde **1,87× a 1,99×** lo de Autores.

Leer no toca su tablón, así que los lectores no hacen fallar los CAS de quien escribe.

Fuente: EXPLICACION.md, Parte 6.1 · docker/rendimiento.csv

Folly rinde muchísimo mejor aquí que en el paper

CON CONTENCIÓN, FRENTE A AUTORES

Paper 0,05×–0,07×

Aquí 0,37×–1,37×

Con n = 1 y u = 50 %, Folly gana: 1,47 frente a 1,07 Mop/s, sin que los rangos se toquen.

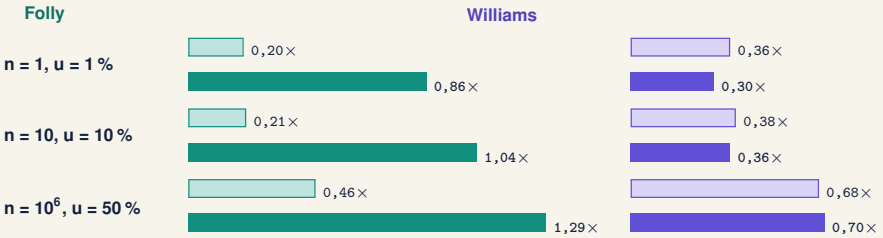
¿Error nuestro? Lo comprobamos

- ✓ **Es su Folly:** el del artefacto, con MD5 comprobado
- ✓ **Sin Docker**, igual: 0,37×–1,40×
- ✓ **Con la receta exacta** (7 × 10 s): 0,36×–1,26×
- ✓ **Compilado de dos formas**, mediana 1,00×

Fuente: EXPLICACION.md, Parte 6.1 · DEFENSA.md, cargo 4 · tablas «Validación de los recortes» y «Docker frente a nativo (binarios oficiales)»

Con un solo hilo, la brecha de Folly ya está ahí

Rendimiento frente a Autores con 1 hilo: sin pelea entre hilos y sin viajes entre núcleos. Barra clara = paper; oscura = réplica.



Williams coincide; Folly no. La causa no es la pelea entre hilos: es algo de la máquina del paper que aquí no se puede reproducir.

Fuente: tablas.md, «Relación con Autores con UN solo hilo» · sin Docker, Folly da 0,88×, 1,04× y 1,24× · 1,00× = rinde igual que Autores

Con un millón de pilas, cuatro algoritmos tocan el mismo techo

En el paper, Autores lidera con 129 Mop/s. Aquí, Autores, Anderson, Libstdc++ y Folly quedan entre **53 y 57**.

Si cuatro algoritmos tan distintos tocan el mismo techo, el límite no está en el algoritmo: con un millón de pilas, casi cada operación va a la **RAM**.

Este PC tiene **un solo módulo**: un canal, 44,8 GB/s como máximo teórico.

NO SUPONER: MEDIR

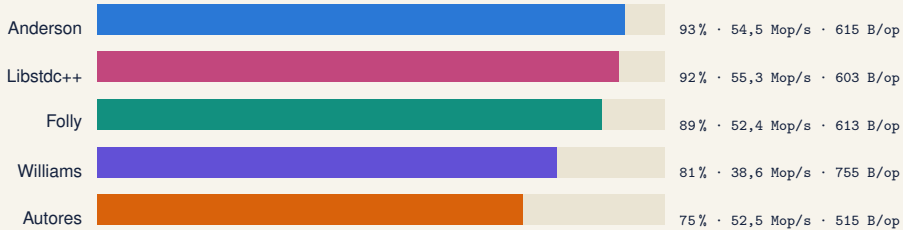
La controladora de memoria trae contadores: un cuentakilómetros en la única carretera al almacén.

Techo leyendo en orden	43 GB/s
Techo al azar, como las pilas	37 GB/s

Fuente: EXPLICACION.md, Parte 6.1 · herramientas/medir_memoria.py, contar_imc.c y ancho_banda.cpp · memoria_ram.csv

Tres algoritmos rozan el techo de la RAM

Tráfico real de RAM como parte del techo al azar (37 GB/s) · 24 hilos, n = 10^6, u = 50 %



Lo refuerza: Anderson pasa de 54,2 a 54,5 Mop/s mientras su tráfico sube de 27 a 34 GB/s.

No lo explica todo: Autores rinde igual usando solo un 75 % del techo.

No lo demuestra: la prueba definitiva sería un segundo módulo de RAM.

Fuente: tablas.md, «Tráfico real de RAM con n = 10^6 y u = 50 %» · figura memoria_ram.png · el fondo de cada barra es el techo

Por debajo del techo, con pocos hilos, Autores va por detrás

n = 10^6, u = 50 %, frente a Autores	Paper, 1 hilo	Aquí, 1 hilo	Aquí, 12 hilos
Anderson	1,03×	1,46×	1,24×
Libstdc++	0,87×	1,44×	1,24×

Los bytes no lo explican

Con 6 hilos, Libstdc++ mueve **más** bytes que Autores (666 frente a 617 por operación) y aun así va 1,38× más rápido.

No es Docker

Sin Docker se repite: 1,33× y 1,40× con 1 hilo, 1,19× los dos con 12. **La causa sigue abierta.**

Fuente: EXPLICACION.md, Parte 6.1 · tablas.md, «Relación con Autores con UN solo hilo»

PARTE 9 · UNOS 11 MINUTOS

Latencia

La figura 6: cuánto tarda cada operación, un candado escondido y por qué con contención casi nada es una victoria.

Las medianas, en nanosegundos

Operación	Autores	Anderson	Folly	Williams	Libstdc++
load, p = 1	14	18	15	34	20
load, p = 24	437	488	473	3 104	3 716
load, p = 56	1 502	1 540	2 977	23 738	61 687
store, p = 1	22	18	17	32	20
store, p = 24	507	483	832	1 021	3 411
store, p = 56	1 516	1 498	4 813	4 105	52 832
store & reclaim, p = 1	28	29	23	44	25
store & reclaim, p = 24	462	470	1 521	821	3 248
store & reclaim, p = 56	813	1 499	6 525	4 439	47 198

En naranja, la mediana más baja de cada fila · Docker, programas oficiales, 5 rondas · Fuente: tablas.md, «Latencia: medianas en ns (figura 6)» · figura fig6_replica.png

Lo que dice el paper, operación por operación

SE REPRODUCE, CON POCO MARGEN

load, siempre el más rápido

Sí en los tres casos. Con contención, por un 8-12 %, y empate con Anderson con p = 56.

NO SE REPRODUCE

store, segundo con p = 1

Aquí es el cuarto de cinco. Son 5 ns, un puñado de ciclos, pero el orden es otro.

EN PARTE

store, el mejor con contención

Empate con Anderson: 507 frente a 483 ns, y 1 516 frente a 1 498.

SE REPRODUCE

store & reclaim, más lento con p = 1

En los cinco algoritmos, con y sin Docker: destruir cuesta.

Fuente: EXPLICACION.md, Parte 6.2 y Parte 7, filas 7 a 10

El candado escondido de Williams

Williams necesita `cmpxchg16b`, que el compilador solo usa con `-mcx16`. El binario oficial de latencia **no la lleva**, y boost::atomic recurrió, sin avisar, a un candado interno.

Fue como cronometrar a un corredor con una mochila que nadie le vio poner.

Prueba: el binario tenía llamadas a lock_pool y ningún cmpxchg16b. Con -mcx16, 0 llamadas.

Williams, ns	Con candado	Sin candado
load, p = 1	52	34
load, p = 24	40 680	3 104
load, p = 56	36 820	23 738
store, p = 24	4 420	1 021
store & reclaim, p = 24	4 594	821

Con el candado, el load con p = 24 parecía **93 veces** más lento que Autores. Sin él, 7 veces. Cambia el veredicto.

Fuente: EXPLICACION.md, Partes 3.3 y 6.2 · tablas.md, «Latencia de Williams: con candado oculto (sin -mcx16) y sin él» · descartadas en docker/latencia_williams_con_candado.csv

PARTE 9 · LATENCIA

Las colas cuentan dos historias que la mediana no ve

ns	Autores	Anderson	Folly	Williams	Libstdc++
store & reclaim, p = 1: p99	29	32	26	49	27
store & reclaim, p = 1: p99,95	30	7 895	36	52	34
load, p = 24: p99,95	1 634	1 408	5 566	10 019	65 230

La caja de Anderson

Vacía su caja cada 2 000 retiradas: 1 de cada 2 000 es el 0,05 %, justo lo que mira el p99,95. De 32 a casi 8 000 ns.

Lock-free frente a wait-free

Con 23 escritores, 1 lectura de cada 2 000 de Williams tarda 10 µs; la de Autores, 1,6 µs: unas 6 veces menos.

Fuente: tablas.md, «Colas de latencia en ns: p50 / p99 / p99,95» · EXPLICACION.md, Parte 6.2 · p99,95 = solo 1 operación de cada 2 000 lo supera

¿Cuánto fiarse? Sin contención, mucho; con contención, poco

Ronda más alta ÷ ronda más baja del mismo punto, caso típico (y peor caso)

Hilos y medida	Docker, oficiales, 5 rondas	Sin Docker, oficiales, 15	Sin Docker, bench.cpp, 5
p = 1, mediana	1,00×	1,00×	1,11×
p = 1, p99	1,00×	1,03×	1,16×
p = 24, mediana	3,40× (13,6×)	5,75× (36,9×)	1,84× (10,4×)
p = 24, p99	3,33× (28,1×)	6,19× (127,3×)	1,50× (36,0×)
p = 56, mediana	1,81× (4,6×)	7,54× (34,1×)	4,32× (30,3×)

Si dos algoritmos difieren un 20 % con p = 24, no significa nada: el mismo cambia más que eso de una ronda a otra.

Fuente: EXPLICACION.md, Parte 6.2 · tablas.md, «Variabilidad de la latencia entre rondas» · 1,00× = el cronómetro siempre marca lo mismo

Sin Docker y con 15 rondas: ¿se sostiene?

Fila del veredicto	Con 15 rondas	Queda
7 · load el más rápido	El más bajo en los 3 casos; con p = 56, Anderson a un 7 %	Se sostiene, por poco
9 · store con contención	461 frente a 470 ns; con p = 56, 1 531, 1 534 y 1 554	Sigue el empate
11 · store & reclaim más rápido	Autores baja de 1 531 a 1 480 ns: un 3 %, dentro del ruido	Ya no se confirma
12 · Williams, más lento	Sin contención, sí; con contención supera a Libstdc++ en 5 de 6	En parte, más débil

Las 5 rondas de Docker de la fila 11 iban de 440 a 1 470 ns: su mediana de 813 pudo depender de la suerte. Es una sospecha, no una medida.

Fuente: EXPLICACION.md, Parte 6.2 · tablas.md, «Latencia: medianas en ns (nativo, binarios oficiales)» · figura nativo_oficial_fig6.png

PARTE 10 · UNOS 10 MINUTOS

Memoria y distancia

¿Quién deja basura? Y, con los mismos 4 hilos, ¿qué pasa si los alejamos?

Anderson retiene un poco; Autores, nada

Hilos	Límite	Autores	Anderson	Folly	Williams	Libstdc++
2	12	9	4 013	9	9	9
4	22	13	6 808	14	15	13
8	42	21	9 910	23	25	21
16	82	36	17 079	39	43	36
24	122	52	19 960	53	59	53

CON 24 HILOS

469 KB

de Anderson, frente a 2,3 KB de Autores

Solo espera la libreta: el objeto muere al instante; su pico de objetos vivos es 14, como los demás.

En bytes es poco: unas 200 veces más que Autores, pero menos de medio mega.

Sin Docker, igual: 20 256 bloques, unos 476 KB.

Pico de bloques de memoria vivos · herramientas/bench_basura.cpp; su «límite» es una fórmula nuestra, no del paper · tablas.md, «Memoria retenida» · figura memoria_retenida.png

Separar 4 hilos cuesta mucho, pero solo donde hay contención

Cambio de rendimiento al pasar de un clúster (35 ns) a cuatro (47 ns), solo núcleos E

Escenario	Autores	Anderson	Folly	Williams	Libstdc++
n = 1, u = 1 %	−32 %	≈ 0 %	+48 %	+25 %	−43 %
n = 1, u = 10 %	−49 %	−33 %	+11 %	−12 %	−41 %
n = 1, u = 50 %	−48 %	−50 %	−9 %	−37 %	≈ −12 %
n = 10, u = 1 %	−54 %	−31 %	−49 %	−34 %	−55 %
n = 10, u = 10 %	−50 %	−39 %	−44 %	−34 %	−50 %
n = 10^6, u = 50 %	≈ +2 %	+2 %	+2 %	+2 %	≈ −2 %

La última fila es el **control**: sin cuaderno compartido, la distancia da igual. Si el castigo apareciera ahí, el montaje estaría mal.

Docker, programas oficiales, 3 rondas · «≈» = rangos solapados · Fuente: tablas.md, «Efecto de separar los hilos entre "sockets"» · figuras afinidad_mapa_4hilos.png y afinidad_dosis_respuesta.png

PARTE 10 · DISTANCIA

Importa compartir caché, no los nanosegundos

Dos hilos de Autores, n = 1, u = 50 %, según dónde caen · Mop/s, mediana por clase



NO ES UNA RAMPA

Es un escalón

En cuanto te levantas de la mesa, da casi igual ir a la sala de al lado o a la del fondo.

El mismo escalón con el programa oficial sin Docker (−44 %) y con bench.cpp (−50 %). Al Mutex le da igual: 3,91 en los dos casos.

Fuente: EXPLICACION.md, Parte 6.4 · tablas.md, «Dos hilos en distintos pares de núcleos» · 14 pares de 4 clases, 3 rondas con Docker y 10 sin Docker

PARTE 10 · DISTANCIA · PREGUNTA ABIERTA

Williams se hunde entre clústeres: la investigación

Con 2 hilos, n = 1, u = 50 %: **3,05 Mop/s** en un clúster y **0,09** en clústeres distintos (−97 %). Sin Docker, 2,93 frente a 0,70 (−76 %).

Sospechoso	Prueba	Resultado
Un par defectuoso	7 pares distintos	Pasa en los 7: descartado
Docker	bench.cpp dentro de Docker, mismos pares	1,12-1,19 Mop/s, sin hundirse: descartado
Docker, otra vez	Programa oficial fuera de Docker, 10 rondas	Se sigue hundiendo, aunque menos
boost::atomic	bench.cpp con la cabecera del artefacto	1,15-1,28 Mop/s, sin hundirse: descartado
El programa oficial	Solo pasa con bench_stack, dentro y fuera	Sigue abierto: sospechoso principal

Dentro y fuera de Docker también cambian Boost (1.74 y 1.83), g++ y glibc. Boost es un candidato natural; no se midió.

Fuente: EXPLICACION.md, Parte 6.4 · control_williams.csv · tablas «Dos hilos en distintos pares de núcleos» y «Williams en bench.cpp, dentro de Docker»

¿Se simularon los 4 Xeon? Solo en parte

SE PARECE

n = 10, u = 1 %

Autores encabeza en los dos a partir del 25 % de la máquina.

LA FORMA SE PARECE

$n = 10^6$, u = 50 %

Rectas que suben en los dos, aunque cambia el orden de las de arriba.

NO SE PARECE

n = 1, u = 1 %

En el paper, Anderson sube y adelanta a Autores. Aquí se queda plano.

Un **simulador de vuelo de juguete**: la física es real y medida, pero el castigo es más suave (1,35× frente a unas 3×), no hay NUMA y hay 16 núcleos E, no 72.

Sirve para ver qué algoritmos son sensibles a la distancia, no para predecir los números del paper.

Fuente: EXPLICACION.md, Parte 6.4 · figura miniatura_frente_a_paper.png (paper y réplica según el porcentaje de núcleos en uso)

PARTE 11 · UNOS 6 MINUTOS

Corrección

Ser rápido no vale de nada si se rompe la memoria. Detectores, reglas rotas a propósito y la cota de reintentos.

Ninguno de los seis rompió la memoria en lo probado

Prueba	Algoritmos	Ejecuciones	Avisos
ASan + UBSan, bench.cpp, 8 y 16 hilos	Autores, Libstdc++, Williams, Mutex	12	0
TSan, bench.cpp, 8 hilos	Los mismos 4	4	0
ASan + UBSan, bench_stack oficial, Docker	Los 5 del paper	10	0
ASan y TSan sobre la versión ingenua	El control: la rota	—	Salta enseguida

«0 avisos» solo vale si el detector estaba puesto: se comprobó la instrumentación, y la versión ingenua lo hace saltar.

Folly y Anderson no pasaron TSan: libfolly.a no está instrumentada y daría avisos falsos.

4 segundos por prueba: un detector solo ve lo que ocurre mientras mira. 0 errores no demuestra que no existan.

ASan: accesos a memoria liberada · TSan: accesos simultáneos sin protección · UBSan: operaciones que C++ no define · Fuente: EXPLICACION.md, Parte 6.5 · herramientas/sanitizers_rivales.sh

Romper las reglas a propósito, para ver si hacían falta

VARIANTE 1

Quitar la salida «otro ya recargó»

`0 errores · 45 intentos`

Con P = 8, sigue siendo correcto, pero **deja de tener cota**: ya no es wait-free.

VARIANTE 2

Poner C = P, violando R-1

`42 348 errores`

Y 4 280 objetos sin liberar. La regla **C ≥ 2P** no es un adorno.

Cada pieza del diseño tiene un trabajo: una garantiza la cota y la otra, que los vales no se acaben.

Fuente: EXPLICACION.md, Parte 6.5 · codigo/boceto/ (boceto didáctico del equipo) · resultados/david-ultra9-285/correccion/

PARTE 11 · CORRECCIÓN

La cota wait-free, contada en el código real

Máximo de intentos en 8 061 142 recargas forzadas (P = 32, C = 64)



8 intentos, como máximo

cota del paper: 32

Variante	Recargas (hasta)	Máx. intentos	Cota	Errores
Real: P = 65 536, C = 524 287	1 158	4	65 536	0
Forzada: P = 32, C = 64	8 061 142	8	32	0
Límite: P = 32, C = 32 (viola R-1)	15 600 117	10	32	0

Fuente: EXPLICACION.md, Parte 6.5 · herramientas/cota_cas.cpp y correr_cota_cas.sh (copia instrumentada del código de los autores, generada al vuelo)

PARTE 12 · UNOS 12 MINUTOS

Sin Docker

Docker no cambia nada, el programa del equipo tenía compartición falsa, y alinear las pilas frena con un millón.

Fuera del contenedor no cambia solo Docker

Pieza	Con Docker	Sin Docker
g++-12	12.3.0	12.4.0
glibc	2.35	2.39
Boost	1.74	1.83
jemalloc	5.2.1	5.3.0
Compilar Folly	getdeps.py	CMake directo

TRES TANDAS SIN DOCKER

Oficiales: 4 660 mediciones

bench.cpp: 2 976

bench.cpp alineado: 2 160

Los oficiales se midieron la noche del 6 al 7 de noviembre de 2026: 4 h 44 min, sin un fallo.

Si algo cambia, no se puede culpar a Docker sin más. Si nada cambia, todas esas piezas juntas tampoco importan.

Fuente: EXPLICACION.md, Parte 6.6 y Parte 8 · pendientes.sh · registro en bin/pendientes.log

Con los programas oficiales, Docker no cambia nada

259/270

puntos de la figura 5 cambian menos de un 10 % (236, menos de un 5 %)

Sin Docker ÷ con Docker	Mediana	Mínimo	Máximo
Autores	1,00×	0,95×	1,11×
Anderson	0,99×	0,95×	1,06×
Folly	1,00×	0,87×	1,06×
Williams	1,01×	0,92×	1,15×
Libstdc++	1,00×	0,87×	1,12×

Antes era una deducción; ahora está medido. Docker no es una máquina virtual: es el mismo kernel y la misma CPU, con una valla alrededor.

Fuente: tablas.md, «Docker frente a nativo (binarios oficiales)» · figura docker_frente_a_nativo_oficial.png (todos los puntos sobre la diagonal)

Con bench.cpp cambian el cronómetro y la pista a la vez

	Oficial, con Docker	bench.cpp, sin Docker
Pilas en memoria	Cada una en su sitio (alignas(128))	Seguidas, sin alinear
Nodos por pila	20 + T/n	16, o 2 si n > 10
Generador de azar	Uno rápido	mt19937_64, más lento
Compilación	g++ 12.3, -O3	g++ 12.4, -O2
Williams	boost::atomic	std::atomic
Autores	Versión del artefacto	Versión publicada
Algoritmos	Los 5 del paper	Sin Folly ni Anderson; con Mutex

CONTROL 1

El **mismo código** dentro y fuera, alternando, 48 puntos:

1,02×

mediana (0,91× a 1,13×). Las diferencias vienen del programa, no de Docker.

Fuente: EXPLICACION.md, Parte 6.6 · controles.sh y herramientas/controles.py · tablas.md, «Mismo código dentro y fuera de Docker»

PARTE 12 · SIN DOCKER · HALLAZGO

Con 10 pilas, bench.cpp sale 3 veces más lento: compartición falsa

Autores, 24 hilos	Oficial, Docker	bench.cpp
n = 1, u = 1 %	19,8	21,0
n = 1, u = 10 %	4,4	5,0
n = 10, u = 1 %	121,9	35,4
n = 10, u = 10 %	55,4	18,4

bench.cpp	Bytes/pila	10 pilas ocupan
Autores	8	~2 líneas
Libstdc++	16	~3 líneas
Williams	16	~3 líneas
Mutex	56	~9 líneas

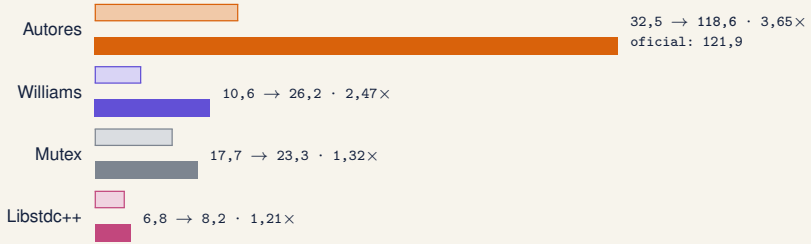
Diez personas, cada una con su cuaderno. . . pero los diez «cuadernos» son **diez renglones de la misma hoja**, y la hoja solo puede estar en una mano.

Con Autores, las «10 pilas» se comportan casi como 2: su diseño compacto juega en su contra.

Mop/s · Fuente: EXPLICACION.md, Parte 6.6, «Control 2» · líneas de caché de 64 bytes

Una sola palabra, alignas(128), y Autores sube 3,65 veces

24 hilos, n = 10, u = 1 % · Mop/s · copia de bench.cpp, medida alternando con la original



El control: con n = 1 no hay vecinas que estorbar, y la palabra no cambia nada: 0,95× a 1,05×.

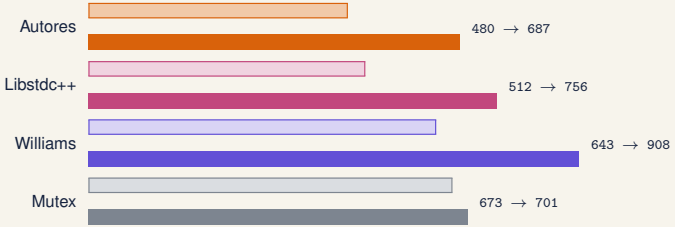
La batería entera, alineada: Autores pasa de 35,4 a 120,3 Mop/s (3,40×). 2 160 mediciones, 0 fallidas.

codigo/bench.cpp no se tocó: corregirlo es una decisión del equipo.

Barra clara = sin alinear; oscura = alineado · Fuente: tablas.md, «Alineación de las pilas en bench.cpp» y «Batería sin Docker con las pilas alineadas» · figura nativo_alineado_fig5.png

Con un millón de pilas, alinear frena

Bytes de RAM por operación, n = 10^6, 24 hilos, u = 50 % · clara = sin alinear; oscura = alineado



POR QUÉ

Cada pila pasa de 8-16 bytes a 128. Un millón: de 8-16 MB, que caben en la L3 de 36 MB, a 128 MB, que no. El fichero ya no cabe en el cajón y hay que ir al almacén.

DOS PREDICCIONES, CUMPLIDAS

n = 10^6: +207 a +265 B por operación y −24 a −27 % de rendimiento. n = 10^5 (12,8 MB, sí cabe): 0,94× a 1,02×.

Medido alternando, 3 rondas, 7 de noviembre de 2026 · Fuente: tablas.md, «Por qué alinear frena con n = 10^6» · herramientas/alinear_memoria.py · nativo_alineado/memoria_alineado.csv

PARTE 12 · SIN DOCKER · BENCH.CPP

La sorpresa del Mutex: el candado simple gana con muchas escrituras

RENDIMIENTO · N = 1, U = 50 %, 24 HILOS

`Mutex 1,86 · Autores 1,15`

Gana de 3 a 36 hilos. Alineado, igual: 1,99 frente a 1,17. Quien espera **duerme**, así que el cuaderno casi no viaja.

LATENCIA · LOAD, P = 24

`mediana 117 ns`

Engaña: 1 de cada 4 operaciones pasa de 5,8 µs, y su p99 llega a 25 µs, frente a 4 µs de Autores.

Wait-free no significa «el más rápido»; significa «nadie se queda atascado». Si quien tiene el candado se detiene, en el Mutex se paran todos.

Fuente: EXPLICACION.md, Parte 6.6 · solo con bench.cpp; el Mutex no está en el paper · la explicación del sueño es razonable, pero no se comprobó

¿Cuánto mueve medir 2 s en vez de 10? Lo medimos

6,7 %
La mitad de los 30 puntos se desvía menos que esto

+19 %
El peor: Folly con n = 10 y u = 10 % (con Docker, hasta +21 %)

4 de 6
escenarios con el orden idéntico; en los otros 2, empates

NO ES RUIDO AL AZAR: ES UN SESGO

Con contención, la versión corta da **más** en 23 de 25 puntos; con n = 10^6, da **menos** en los 5. Como un cronómetro que adelanta en una pista y atrasa en otra. Por qué, no se midió.

Regla que sale de aquí: una ventaja del 10 % no demuestra nada; una de 2 o 3 veces, sí.

Receta exacta (7 × 10 s) frente a 2 s × 5, sin Docker, 6 escenarios, 30 puntos · Fuente: EXPLICACION.md, Parte 6.7 · tablas.md, «Validación de los recortes»

PARTE 13 · UNOS 9 MINUTOS

Veredicto

Las 15 afirmaciones del paper, una por una; lo que no pudimos hacer, y lo que descubrimos por el camino.

Afirmaciones 1 a 8

#	Lo que dice el paper	Resultado aquí	Veredicto
1	Mucho más rápido que Libstdc++	Gana con contención; empata con $n = 10^6$	Se reproduce
2	Mucho más rápido que Williams	Gana en los 6 escenarios, 0,24×–0,73×	Se reproduce
3	Mucho más rápido que Folly	Gana 3, empata 2 y pierde 1	No se reproduce
4	Con u pequeño, supera a Anderson	+6 % con $n = 1$ y +65 % con $n = 10$	Se reproduce
5	Con n grande, supera a Anderson	Pierde hasta 18 hilos; empata desde 24	No, en este PC
6	Con mucha contención, gana Anderson	Anderson 1,87×–1,99× con $n = 1$	Se reproduce
7	load: siempre la menor latencia	La más baja en los 3 casos; por poco	Sí, poco margen
8	store: segundo con $p = 1$, tras Anderson	Cuarto de cinco, por 5 ns	No se reproduce

Cifras con Docker; sin Docker, las mismas conclusiones · Fuente: EXPLICACION.md, Parte 7 · DEFENSA.md, sección 4

Afirmaciones 9 a 15

#	Lo que dice el paper	Resultado aquí	Veredicto
9	store: el mejor con contención	Empate con Anderson	En parte
10	store & reclaim, más lento con p = 1	En los cinco, con y sin Docker	Se reproduce
11	store & reclaim, a veces más rápido	Con 15 rondas, solo un 1-4 %: ruido	No se confirma
12	Williams, casi siempre más lento	Sin contención, sí; con contención, no siempre	En parte
13	Anderson retiene algo; Autores, nada	19 960 bloques (469 KB) frente a 52	Se reproduce
14	La recarga reintenta $\leq$ P veces	Máximo 8 con P = 32	Se reproduce
15	No hay use-after-free	0 errores con ASan y TSan	Sí, en lo probado

La fila 11 pasó de «se reproduce» a «no se confirma» con los datos de 15 rondas. Si el equipo prefiere «dudoso», es cambiar una palabra.

Fuente: EXPLICACION.md, Parte 7 · la causa que propone el paper para la fila 11 tampoco se comprobó

PARTE 13 · VEREDICTO

La idea central sale bien parada



9 se reproducen: 1, 2, 4, 6, 7, 10, 13, 14, 15

2 en parte: 9, 12

3 no: 3, 5, 8

1 no se confirma: 11

DICHO EN UNA FRASE

El algoritmo es correcto, cumple su cota, no acumula basura y lee rápido. No se sostiene la superioridad frente a Folly ni con muchísimas pilas, y la escala de 4 sockets no se pudo reproducir.

Sin Docker, con los mismos programas oficiales, la tabla sale igual salvo la fila 11 · Fuente: EXPLICACION.md, Parte 7

Lo que esta réplica no puede decir

La máquina

- Sin 4 sockets: castigo 1,35× frente a unas 3×
- Sin NUMA: numactl aquí no hace nada
- Sin SMT; 24 núcleos, no 72
- Un solo módulo de RAM
- CPU híbrida: núcleos P y E
- Frecuencia sin fijar; máximo 99 °C

El método

- Recortes: hasta 19-21 %, con sesgo
- Valores del paper sacados de sus gráficos
- Williams en latencia, con binario aparte
- Dos intentos descartados por errores nuestros
- PC de escritorio, no aislado
- La primera medición de RAM casi nos engaña

Las conclusiones

- Cuatro preguntas siguen abiertas
- El techo de la RAM, reforzado, no demostrado
- La latencia con contención es ruidosa
- bench_basura y su límite son nuestros
- TSan no se pasó a Folly ni a Anderson

90 °C o más: 5 de 3 720 ejecuciones con Docker y 26 de 9 796 sin Docker, repartidas y conservadas · Fuente: EXPLICACION.md, Parte 8 · DEFENSA.md, cargos 1, 5, 9 y 12

Cuatro preguntas abiertas, y cómo cerrarlas

¿Por qué Folly va tan distinto?

Ya pasa con 1 hilo. **Cerrarla:** escribir a los autores; comprobar la versión de Folly del artefacto; medir con perf.

¿Por qué se hunde Williams?

Solo en el programa oficial, entre clústeres. **Cerrarla:** bench_stack con Boost 1.74 fuera de Docker, o con 1.83 dentro.

¿Por qué Autores va detrás con pocos hilos?

Con $n = 10^6$; los bytes no lo explican. **Cerrarla:** contadores de hardware, perf.

¿Es la RAM el techo?

Tres algoritmos al 89-93 %. **Cerrarla:** un segundo módulo DDR5-5600 y repetir medir_memoria.py.

Fuente: EXPLICACION.md, Partes 0 y 8 · DEFENSA.md, sección 5 (plan de mejoras)

Siete hallazgos que no estaban en el paper

#	Hallazgo	La cifra
1	bench.cpp del equipo tiene compartición falsa	Autores, de 35 a 120 Mop/s al alinear
2	El binario oficial de latencia se compila sin -mcx16	Williams: 40 680 → 3 104 ns
3	Importa compartir caché, no la distancia	−45 % al salir del clúster; 47 o 86 ns, igual
4	Williams se hunde entre clústeres en el programa oficial	−97 % con Docker, −76 % sin Docker
5	Con un millón de pilas, el freno es sobre todo la RAM	89-93 % del techo de 37 GB/s
6	Alinear frena con un millón: dejan de caber en la L3	+207 a +265 B de RAM por operación
7	Docker no cambia los programas oficiales	259 de 270 puntos, menos de un 10 %

Los dos primeros nacen de errores, uno del programa del equipo y otro del artefacto, y salieron de controles diseñados para buscarlos.

Fuente: EXPLICACION.md, Parte 0, «Lo más valioso que salió por el camino» · Parte 6.6

PARTE 14 · UNOS 7 MINUTOS

Defensa

Un tribunal simulado con 14 críticas: qué responder, qué conceder y qué mejorar.

Las 14 críticas que nos hicimos, y su fallo

#	Cargo	Fallo
1	Sin 4 sockets no es réplica	Concedido en parte
2	La simulación es un decorado	A favor, con matiz
3	Recortaron las mediciones	A favor, con matiz
4	Folly falla por un error suyo	A favor
5	La latencia es puro ruido	Concedido en parte
6	Docker contamina las medidas	A favor
7	Se equivocaron tres veces	A favor

#	Cargo	Fallo
8	«0 errores» no demuestra nada	Concedido
9	Solo midieron su RAM	Concedido en parte
10	Eligieron lo que convenía	A favor
11	Sin intervalos de confianza	Concedido en parte
12	La máquina no estaba controlada	Concedido en parte
13	¿Lo hicieron ustedes o una IA?	Depende del equipo
14	El informe contradice los datos	En contra, si no se corrige

Un jurado se fía más de quien reconoce un límite que de quien lo esconde: **conceder** también es defender.

La sentencia: una reproducción válida y honesta a escala de un PC

Criterio	Valoración	Por qué
Artefactos disponibles	Sí	CSV, figuras y guiones en el repositorio
Funcionan	Sí	Una orden por experimento; se reanudan solos
Reutilizables	En parte	Las listas de núcleos son de esta CPU
Resultados	En parte, bien argumentado	9 de 15, más 2 en parte; lo central se sostiene
Transparencia	Alta	Errores propios y datos descartados a la vista
Alcance	Limitado, y declarado	Nada sobre 4 sockets ni NUMA

Más fuertes: el argumento de Folly con un hilo, y haber medido en vez de suponer: Docker, recortes, RAM y distancia.

Más débiles: la latencia con contención y, si no se corrigen antes, las erratas del informe del equipo.

Criterios inspirados en las insignias de artefactos de la ACM · Fuente: DEFENSA.md, sección 4

Qué haríamos con más tiempo, por orden de rendimiento

Con una semana

- **Segundo módulo de RAM** y repetir n = 10^6 (cargo 9)
- Intervalos de confianza por bootstrap: unas 30 líneas en analizar.py (cargo 11)
- Figura 5 entera con la receta exacta: una noche (cargo 3)
- Latencia con frecuencia fija y núcleos aislados (cargo 5)
- Que otro miembro repita una parte en su PC (cargo 7)
- Escribir a los autores sobre Folly (cargo 4)

Más lejos

- Una máquina de 2 o 4 sockets: servidor de la universidad o nube (cargos 1 y 2)
- Un AMD Ryzen con dos chiplets, midiendo antes su mapa (cargos 1 y 2)
- TSan en Folly y Anderson, con Folly instrumentado (cargo 8)
- Comprobación de modelos, por ejemplo con GenMC, con P y C pequeños (cargo 8)
- Contadores de hardware con perf (cargos 4 y 9)

Entre paréntesis, la crítica del tribunal que cada mejora debilita · Fuente: DEFENSA.md, sección 5

PARTE 15 · UNOS 5 MINUTOS

Recursos

Cómo repetirlo, dónde está cada dato, las referencias, un glosario y las preguntas que conviene llevar ensayadas.

Cómo repetirlo todo: una orden por experimento

```
cd replica/completa
bash correr_docker.sh                    # los 5 con Docker · unas 3 h
SOLO_DESCARGA=1 bash preparar.sh         # sin Docker, sin instalar nada
bash correr_nativo.sh equipo             # bench.cpp · unas 1,5 h
bash pendientes.sh                       # oficiales y alineado · unas 5 h
python3 herramientas/comprobar_final.py # comprueba la tanda, sin IA
python3 herramientas/medir_memoria.py    # tráfico de RAM · unos 20 min
bash controles.sh                        # controles · unos 20 min
python3 analizar.py                      # tablas y figuras · unos 4 s
```

Se reanudan solas: si algo se corta, se relanza la misma orden y salta lo ya medido.

Una medición a la vez, con la máquina libre; y no editar un .sh mientras corre.

Otra CPU: medir antes el mapa de latencias y adaptar las listas de núcleos de baterias.py.

Fuente: replica/completa/README.md y EXPLICACION.md, Parte 9 · correr_docker.sh usa sudo docker; con permisos propios: DOCKER=docker

Dónde está cada dato

resultados/david-ultra9-285/	Qué contiene
docker/	Binarios oficiales con Docker
nativo_oficial/	Binarios oficiales sin Docker y receta exacta
nativo/	bench.cpp del equipo sin Docker
nativo_alineado/	bench.cpp alineado y su tráfico de RAM
memoria_ram.csv	Tráfico real de RAM y techo de la máquina
correccion/	ASan, TSan, boceto y cota de CAS
topologia/	Latencia entre cada par de núcleos
tablas.md · figuras/	Todas las tablas y las 20 gráficas

PARA LEER, EN ESTE ORDEN

EXPLICACION.md: todo, desde cero; Partes 0, 7 y 8 para empezar

DEFENSA.md: el tribunal, la chuleta de cifras y las 20 preguntas

README.md: qué hace cada archivo y cuánto tarda

LEEME.md: cada CSV y cada columna

Las tres primeras están en replica/completa/ y LEEME.md en replica/resultados/david-ultra9-285/ · artefacto/, artefacto_docker/ y bin/ no se versionan

Referencias: el paper, su artefacto y los rivales

de Wolff, I. G.; Anderson, D.; Keller, G. K.; Seletskiy, A. **A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting.** Euro-Par 2024, LNCS 14803.
Artefacto: doi.org/10.5281/zenodo.11775182 · MD5 e7e03425dfd426385a769788e5460fcd

Algoritmo	Fuente
Autores	github.com/ivogabe/wait-free-atomic-shared-pointer
Libstdc++	std::atomic<std::shared_ptr> de GCC, C++20 (bits/shared_ptr_atomic.h)
Williams	Williams, A. C++ Concurrency in Action. Manning, 2012 · github.com/anthonywilliams/atomic_shared_ptr
Folly	github.com/facebook/folly · commit 8a9b2220d5946e5db8c8e0188332df4990e80b08
Anderson	Anderson, Blelloch y Wei, PLDI 2021 y PLDI 2022 · Anderson, CppCon 2023 · código ParlayLib, en el artefacto

Fuente: EXPLICACION.md, «Referencias y fuentes» · el binario oficial usa la versión de Autores del artefacto, anterior a la del repositorio

Para profundizar

Teoría

Herlihy, «Wait-free synchronization», ACM TOPLAS, 1991

Michael, «Hazard pointers», IEEE TPDS, 2004

Sutter, «Atomic smart pointers», WG21 N4058, 2014

Chen et al., use-after-free, RAID 2023

Hardware (dato indirecto)

Intel Community: 40-45 ns dentro del socket

Chips and Cheese: 130-140 ns entre sockets

Technical City: ficha del Xeon E7-8867

Reproducibilidad

ACM, Artifact Review and Badging: repetición, reproducción y réplica

Herramientas

AddressSanitizer · ThreadSanitizer · jemalloc · Google Benchmark v1.8.3 · GenMC

Fuente: EXPLICACION.md, «Referencias y fuentes» · DEFENSA.md, sección 2 · los números entre corchetes del paper siguen su bibliografía

Dónde están los materiales del equipo

La réplica

github.com/davichiacustico-ux/replica-wait-free-refcount

Cambios de la réplica completa: pull request #4

Guiones, CSV, tablas, 20 figuras, EXPLICACION.md y DEFENSA.md

Guías y podcast

github.com/dtamotu/guias-wait-free-refcount

podcast_notebooklm/: 14 episodios para NotebookLM, con sus fuentes y prompts

Las guías anteriores a la réplica pueden contradecirla: si no coinciden, manda EXPLICACION.md

FIGURAS PARA ENSEÑAR EN LAS PREGUNTAS

`fig5_paper_frente_a_replica.png` · `afinidad_dosis_respuesta.png` · `docker_frente_a_nativo_oficial.png` · `memoria_ram.png` · `nativo_alineado_fig5.png` · `topologia.png`

Las figuras están en replica/resultados/david-ultra9-285/figuras/ · DEFENSA.md, sección 5, «Diapositivas de reserva»

Glosario de bolsillo

Afinidad	Fijar en qué núcleo corre un hilo
Atómica	Ocurre entera o no ocurre
Bloque de control	La libreta: donde vive el contador
C	Máximo de vales en el tablón
CAS	«Si vale X, ponlo a Y»; puede fallar
Compartición falsa	Datos distintos en la misma línea
Contención	Muchos hilos peleando por un dato
Contador partido	Cuenta junto al puntero y en la libreta
DWCAS	CAS sobre 16 bytes (cmpxchg16b)
fetch_add	Suma atómica; nunca falla
Hazard pointer	Cartel: «estoy usando esto»
Línea de caché	Bloque de 64 bytes que viaja
Lock-free	Alguien avanza; tú quizá no
Mop/s	Millones de operaciones por segundo
n · u	Número de pilas · % de escrituras
NUMA	Parte de la RAM está más lejos
P · p	Máximo de hilos · hilos en latencia
p50 · p99	Mediana · lo malo que puede ponerse
Peso	Los vales que quedan en el tablón
Recarga	Añadir vales: increase_weight
Socket	Chip físico; el paper usa 4
τ de Kendall	De -1 a $+1$: ¿coincide el orden?
Use-after-free	Usar memoria ya liberada
Wait-free	Todos acaban en pasos acotados

Glosario completo: EXPLICACION.md, Parte 10

Respuestas cortas: método

¿Por qué no una máquina virtual con 4 nodos?
Sería un decorado: ningún viaje tardaría más. Preferimos distancias reales, medidas.

¿Por qué dividieron los hilos entre 6?
Para conservar la proporción: 144 era toda la máquina del paper; 24 es toda esta.

¿Por qué 2 s y no 10?
Para que cupiera en una hora en vez de cinco. Lo medimos: el orden se mantiene en 4 de 6.

¿Por qué orden aleatorio, un proceso por punto?
Para que el calor o una interrupción afecten a todos por igual.

¿Usaron el código de los autores?
Sí: el artefacto de Zenodo, con MD5 comprobado. bench.cpp es una segunda opinión.

¿Qué es wait-free y cómo lo comprobaron?
Pasos acotados pase lo que pase. Contamos intentos: 8 como mucho, con cota 32.

¿Por qué Folly sale tan distinto?
No lo sabemos. No es contención (pasa con 1 hilo) ni el montaje (Williams coincide).

¿Por qué empatan con un millón de pilas?
Muy probablemente la RAM: tres algoritmos al 89-93 % del techo de un solo módulo.

Fuente: DEFENSA.md, sección 6 («Ensayo: 20 preguntas del jurado») · cada respuesta remite a su parte de EXPLICACION.md

Respuestas cortas: resultados

¿Qué aporta su réplica?
Williams en latencia, el candado sin -mcx16, la batería de afinidad y la RAM medida.

¿Influye Docker?
No: de 270 puntos, 259 cambian menos de un 10 %.

¿Cómo saben que ASan funcionaba?
La versión ingenua lo hace saltar, y los binarios estaban instrumentados.

¿Qué es la compartición falsa?
Renglones distintos de la misma hoja: datos distintos, misma línea de caché.

¿Por qué alinear frena con un millón?
Las pilas pasan a 128 bytes y no caben en la L3: +207 a +265 B de RAM por operación.

¿Por qué se hunde Williams entre clústeres?
No lo sabemos. Descartamos un par defectuoso, Docker y boost::atomic.

¿Por qué Anderson gana con escrituras?
En Autores leer modifica el tablón y hace fallar los CAS; en Anderson, no.

¿Cuánta memoria retiene Anderson?
Hasta 19 960 bloques, 469 KB, con 24 hilos. Autores, nada sobre el límite.

Fuente: DEFENSA.md, sección 6 · cifras de la chuleta de DEFENSA.md, sección 8

PARA CERRAR

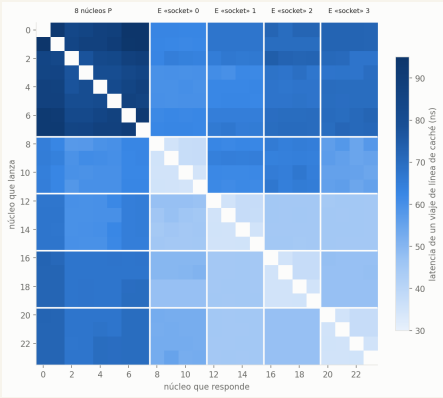
Un solo fetch_sub que nunca falla resuelve la carrera entre leer y liberar. En un PC de 24 núcleos, la idea aguanta; las cifras dependen de la máquina.

Gracias. ¿Preguntas?

David Tamo Turpo · Jheremy Manuel Alvarez Astete
Diego Quispe Candelario

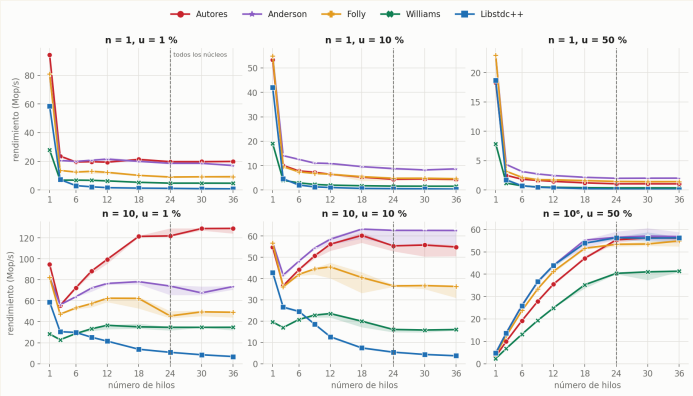
ANEXO · FIGURA 1 DE 20

Latencia entre cada par de núcleos

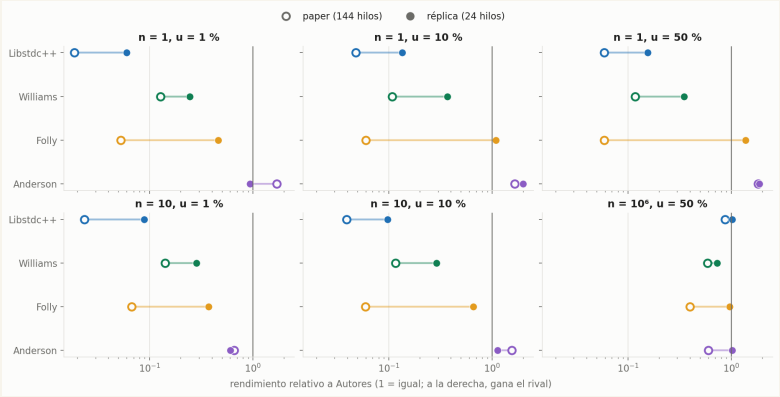


figuras/topologia.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 5.2

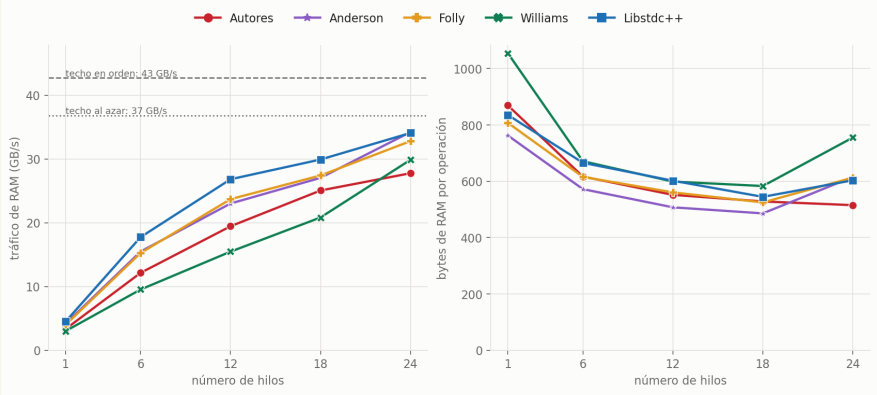
Rendimiento de las 5 implementaciones en los 6 escenarios



Cuánto rinde cada rival respecto a Autores, en el paper y en la réplica

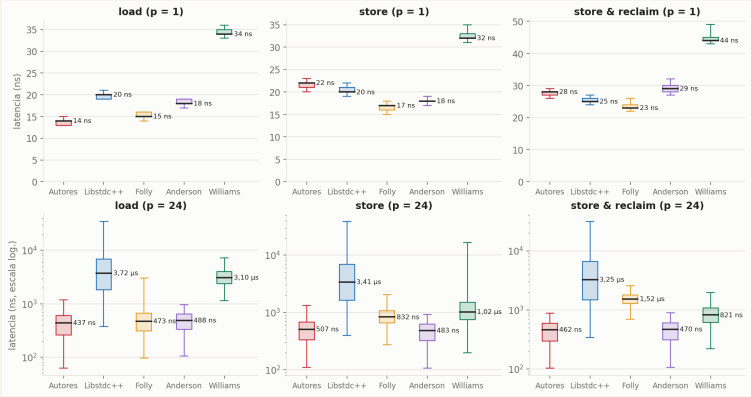


Tráfico real de RAM

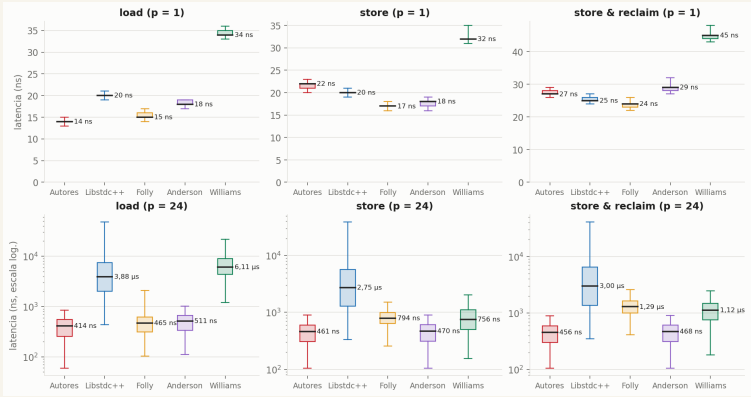


figuras/memoria_ram.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.1

Latencia de cada operación con y sin contención

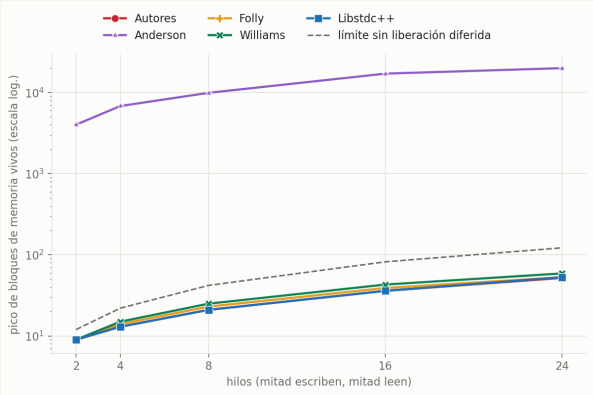


Latencia sin Docker con los programas oficiales



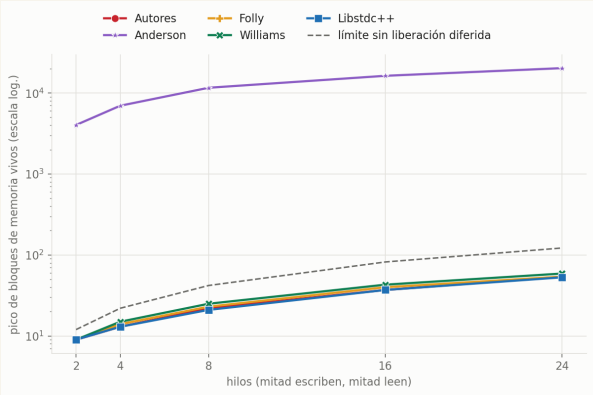
ANEXO · FIGURA 7 DE 20

Pico de bloques de memoria vivos



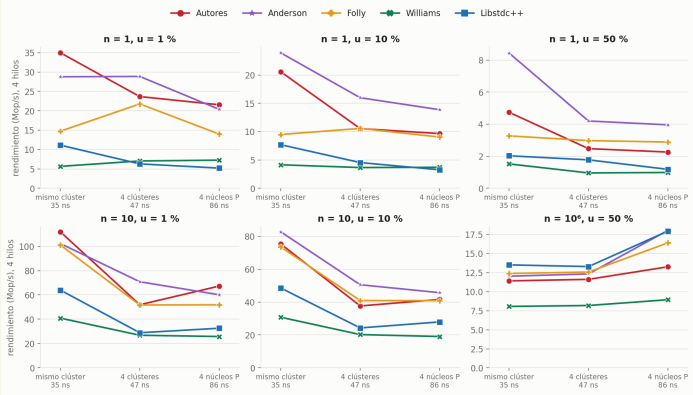
figuras/memoria_retenida.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.3

Pico de bloques de memoria vivos, sin Docker

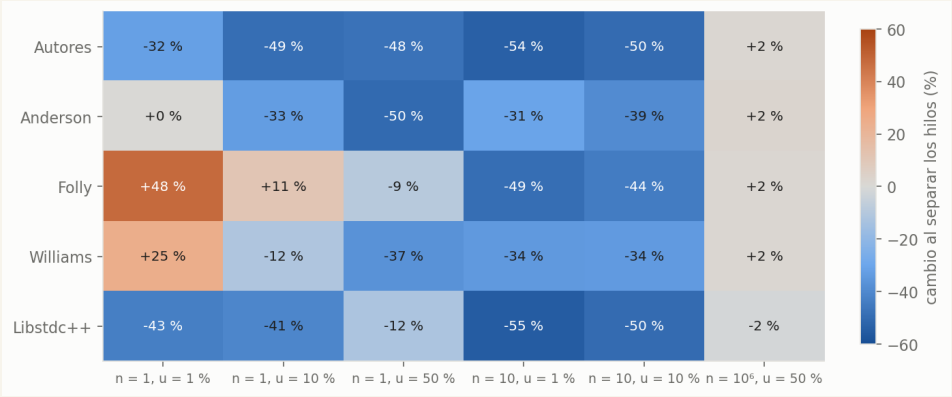


`figuras/nativo_oficial_memoria_retenida.png` · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.3

Rendimiento con 4 hilos a distancia creciente

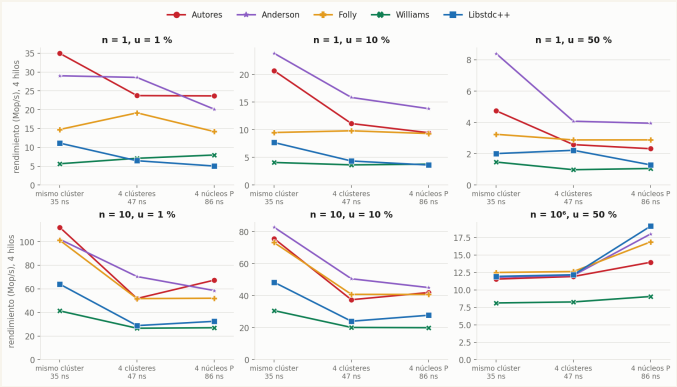


Cambio de rendimiento al separar 4 hilos



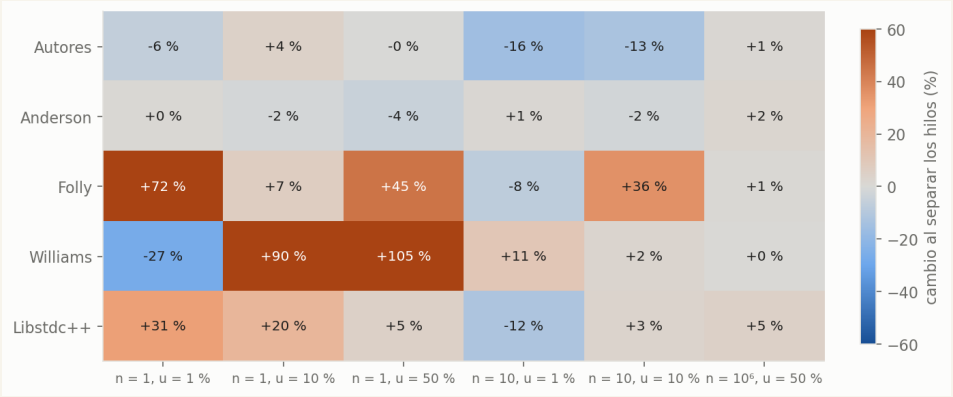
ANEXO · FIGURA 11 DE 20

Rendimiento con 4 hilos a distancia creciente, sin Docker y con los programas oficiales



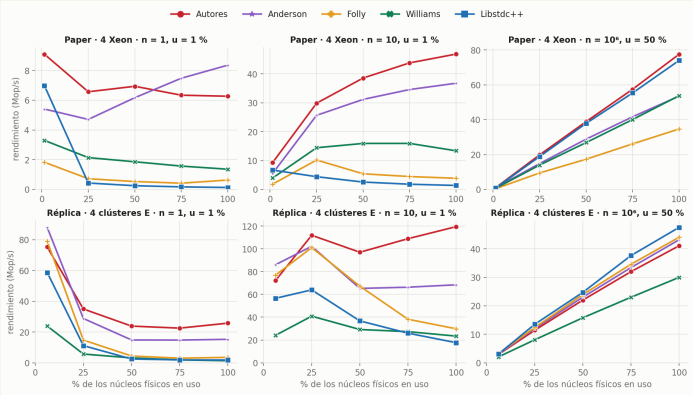
figuras/nativo_oficial_dosis_respuesta.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.4

Cambio de rendimiento al separar 8 hilos



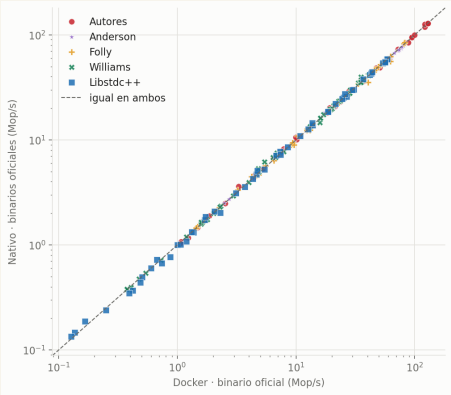
`figuras/afinidad_mapa_8hilos.png` · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.4

Paper y réplica según el porcentaje de núcleos en uso



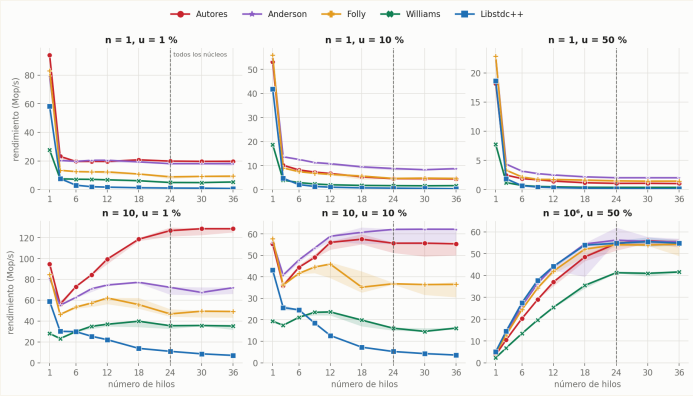
figuras/miniatura_frente_a_paper.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.4

Cada punto de la figura 5 con los programas oficiales, con Docker y sin Docker



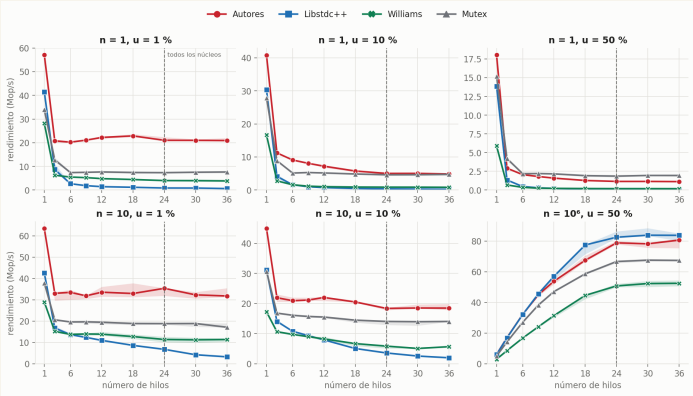
figuras/docker_frente_a_nativo_oficial.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.6

Rendimiento sin Docker con los programas oficiales



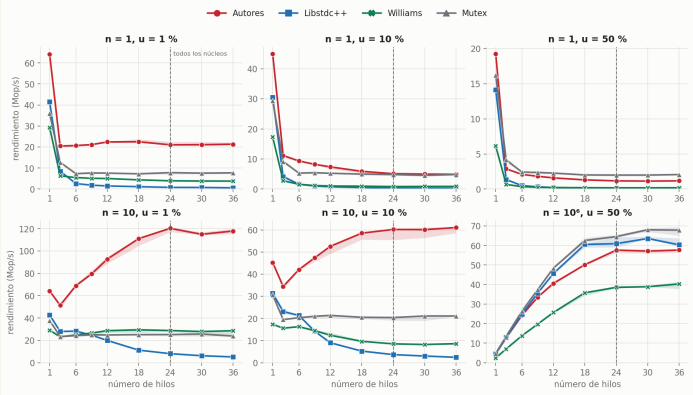
figuras/nativo_oficial_fig5.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.6

Rendimiento sin Docker con bench.cpp del equipo

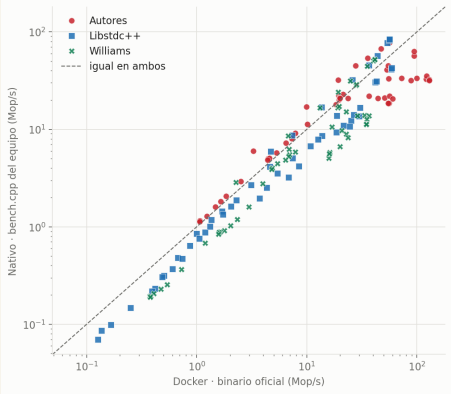


figuras/nativo_fig5.png · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.6

Rendimiento sin Docker con bench.cpp y las pilas alineadas

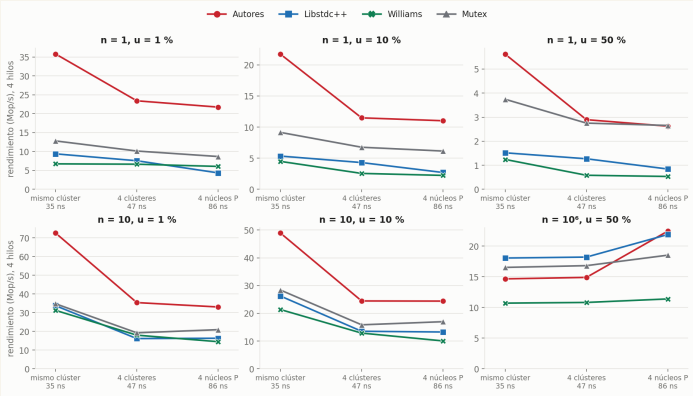


Cada punto de la figura 5, con Docker y sin Docker



`figuras/docker_frente_a_nativo.png` · generada por analizar.py · se explica en EXPLICACION.md, Parte 6.6

Rendimiento con 4 hilos, de más cerca a más lejos, sin Docker



ANEXO · FIGURA 20 DE 20

Latencia sin Docker con bench.cpp del equipo

